

DBMS PLACEMENT PREPARATION NOTES

Modules 10–13 • Subqueries • Normalization • Functional Dependencies • Transactions

Study approach: These notes explain the terminology first, then give examples, placement traps, and quick revision points. The aim is understanding + interview/MCQ readiness.

Sequence continued: Modules 1–9 covered DBMS fundamentals, ER model, keys, relational concepts, SQL, functions and joins. These are the next four topics in the placement sequence.

MODULE 10 — SUBQUERIES / NESTED QUERIES

Subquery: A query written inside another SQL query. The inner query is executed to provide a value or set of values used by the outer query.

Think of it as: **Inner query** → **produces result** → **Outer query uses that result**.

Basic structure

```
SELECT column FROM table WHERE column IN (SELECT column FROM table WHERE condition);
```

1. Why use a subquery?

Use a subquery when the answer to one condition depends on the result of another query.

Example: Find employees whose salary is greater than the average salary.

```
SELECT name, salary FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

2. Single-Row Subquery

Returns exactly one row/value. Common operators: =, >, <, >=, <=, <>.

```
SELECT name FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

3. Multi-Row Subquery

Returns multiple values. Common operators: IN, ANY, ALL.

```
SELECT name FROM Employee WHERE dept_id IN (SELECT dept_id FROM Department WHERE location = 'Hyderabad');
```

4. IN

IN: Checks whether a value matches any value returned by the subquery.

5. ANY / SOME

ANY/SOME: Condition becomes true if it is satisfied for at least one value returned by the subquery.

6. ALL

ALL: Condition must be satisfied for every value returned by the subquery.

7. Correlated Subquery

Correlated subquery: Inner query depends on a value from the outer query, so it is evaluated in relation to each outer row.

```
SELECT e.name FROM Employee e WHERE e.salary > (SELECT AVG(e2.salary) FROM Employee e2 WHERE e2.dept_id = e.dept_id);
```

Memory: **Normal subquery** → **independent result** | **Correlated subquery** → **depends on outer row**.

8. Subquery in SELECT / FROM / WHERE

Location	Use
WHERE	Filter rows using another query's result.

Location	Use
FROM	Use a subquery as a derived table/result set.
SELECT	Return a calculated value produced by a subquery.

9. EXISTS

EXISTS: Tests whether the subquery returns at least one row.

```
SELECT name FROM Employee e WHERE EXISTS (SELECT 1 FROM Department d WHERE d.dept_id = e.dept_id);
```

10. IN vs EXISTS

IN	EXISTS
Compares a value with values returned by the subquery.	Checks whether the subquery returns at least one row.
Useful when you need membership.	Useful when you care about existence of a matching row.

■ Placement Questions

- What is a nested query? → A query inside another query.
- Which operators are common with multiple-row subqueries? → IN, ANY, ALL.
- What makes a subquery correlated? → It refers to a column/value from the outer query.
- What does EXISTS check? → Whether at least one row exists.

MODULE 11 — DATABASE NORMALIZATION

Normalization: Process of organizing data to reduce redundancy and avoid data anomalies.

Redundancy: Same information unnecessarily repeated in multiple rows.

Anomaly: An unwanted problem caused by poor table design.



Remove repeating groups → partial dependency → transitive dependency

1. Why Normalization is Needed

Main goals: reduce duplicate data, improve consistency, and make insert/update/delete operations safer.

2. Types of Anomalies

Anomaly	Meaning
Insertion anomaly	Cannot insert a fact without also inserting unrelated data.
Update anomaly	Same fact appears in multiple rows and must be updated everywhere.
Deletion anomaly	Deleting one fact accidentally removes another useful fact.

3. First Normal Form — 1NF

1NF: Each column contains atomic values, and there are no repeating groups.

Atomic: A value that is treated as a single value in the table.

Bad idea: Phone = '9876, 8765' in one cell. Better: store each phone value separately or model multiple phones appropriately.

4. Second Normal Form — 2NF

2NF: Table is in 1NF and every non-key attribute depends on the whole candidate key, not just part of a composite key.

Partial dependency: A non-key attribute depends on only part of a composite key.

Important: If the primary key has only one attribute, partial dependency cannot occur.

5. Third Normal Form — 3NF

3NF: Table is in 2NF and non-key attributes do not depend on other non-key attributes.

Transitive dependency: A non-key attribute depends on another non-key attribute.

Example: Student_ID → Dept_ID and Dept_ID → Dept_Name, so Student_ID → Dept_Name indirectly. This is a transitive dependency.

6. BCNF — Boyce-Codd Normal Form

BCNF: For every non-trivial functional dependency $X \rightarrow Y$, X must be a super key.

For placement preparation, remember: **BCNF is stronger than 3NF.**

7. 1NF vs 2NF vs 3NF

Normal Form	Main condition
1NF	Atomic values; no repeating groups.
2NF	1NF + no partial dependency.
3NF	2NF + no transitive dependency.
BCNF	Every determinant is a super key.

8. Easy Memory Chain

1NF → Atomic | 2NF → No Partial Dependency | 3NF → No Transitive Dependency | BCNF → Determinant must be Super Key

■ Placement Trap

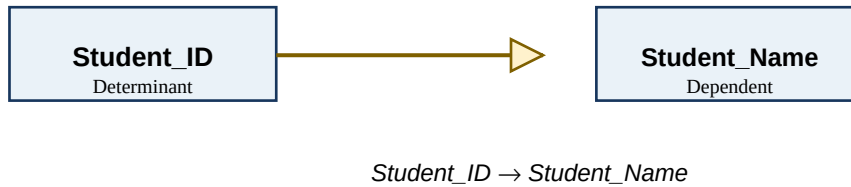
2NF mainly matters when there is a **composite key**. Do not say '2NF simply means no duplicate data'—that is not its definition.

Normalization reduces redundancy, but over-normalization can increase the number of tables and joins. In real systems, denormalization may sometimes be used for performance.

MODULE 12 — FUNCTIONAL DEPENDENCIES

Functional Dependency (FD): A relationship where one set of attributes determines another set of attributes.

Written as: $X \rightarrow Y$ and read as 'X determines Y'.



Example: If each Student_ID identifies exactly one Student_Name, then $Student_ID \rightarrow Student_Name$.

1. Determinant

Determinant: The attribute(s) on the left side of a functional dependency.

In $X \rightarrow Y$, **X is the determinant** and Y is the dependent attribute.

2. Dependent Attribute

The attribute(s) on the right side of a functional dependency are determined by the left side.

3. Full Functional Dependency

Full dependency: A non-key attribute depends on the entire composite key, not a part of it.

This concept is important for understanding **2NF**.

4. Partial Dependency

Partial dependency: A non-key attribute depends on only part of a composite key.

Partial dependency violates **2NF**.

5. Transitive Dependency

Transitive dependency: A non-key attribute depends on another non-key attribute through the key.

Example: $Student_ID \rightarrow Dept_ID$ and $Dept_ID \rightarrow Dept_Name$. Therefore Student_ID indirectly determines Dept_Name.

Transitive dependency is the main dependency removed when moving toward **3NF**.

6. Trivial Functional Dependency

$X \rightarrow Y$ is trivial if Y is a subset of X.

Example: $\{Student_ID, Name\} \rightarrow Name$.

7. Non-Trivial Functional Dependency

$X \rightarrow Y$ is non-trivial when Y is not a subset of X.

8. Armstrong's Axioms — Placement Level

Armstrong's axioms are inference rules used to derive additional functional dependencies.

Rule	Meaning
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$.
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$.

Rule	Meaning
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Memory: Reflexivity $\rightarrow$ Augmentation $\rightarrow$ Transitivity = **RAT**.

9. Why Functional Dependencies Matter

FDs help identify candidate keys and analyze whether a relation satisfies normalization rules.

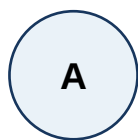
■ Placement Questions

- What is X in $X \rightarrow Y$? $\rightarrow$ Determinant.
- Which dependency is related to 2NF? $\rightarrow$ Partial dependency.
- Which dependency is related to 3NF? $\rightarrow$ Transitive dependency.
- What does $X \rightarrow Y$ mean? $\rightarrow$ X determines Y.

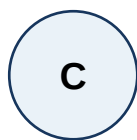
MODULE 13 — TRANSACTIONS & ACID PROPERTIES

Transaction: A sequence of one or more database operations treated as one logical unit of work.

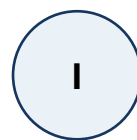
Example: Bank transfer — debit ₹500 from A and credit ₹500 to B should be treated as one transaction.



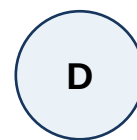
Atomicity



Consistency



Isolation



Durability

1. Transaction States

State	Meaning
Active	Transaction is executing operations.
Partially Committed	Last statement has executed, but final commit is not yet completed.
Committed	All changes are permanently saved.
Failed	Transaction cannot continue because of an error/failure.
Aborted	Transaction has been rolled back; its changes are undone.

2. ACID Properties

Property	Meaning
Atomicity	All operations happen, or none happen.
Consistency	Transaction moves the database from one valid state to another valid state.
Isolation	Concurrent transactions should not incorrectly interfere with each other.
Durability	Committed changes survive failures.

3. COMMIT

COMMIT: Permanently saves the changes made by the current transaction.

COMMIT;

4. ROLLBACK

ROLLBACK: Undoes changes that have not been committed.

ROLLBACK;

5. SAVEPOINT

SAVEPOINT: Marks a point inside a transaction so you can roll back to that point instead of undoing the entire transaction.

SAVEPOINT sp1;

ROLLBACK TO sp1;

6. Concurrency Control — Why Needed?

When multiple transactions execute at the same time, incorrect results can occur unless the DBMS controls their interaction.

7. Common Concurrency Problems

Problem	Meaning
Dirty Read	A transaction reads data written by another transaction before it is committed.
Non-repeatable Read	The same row read twice gives different committed values because another transaction updated it.
Phantom Read	Repeating a range query returns a different set of rows because another transaction inserted/deleted rows.

8. Serial vs Concurrent Schedule

Serial: Transactions execute one after another without interleaving.

Concurrent: Operations of multiple transactions may be interleaved.

9. Deadlock — Basic Concept

Deadlock: Two or more transactions wait for resources held by each other, so none can proceed.

Example: T1 holds A and waits for B; T2 holds B and waits for A.

■ Placement Memory

Atomicity → All or Nothing

Consistency → Valid State to Valid State

Isolation → Transactions Don't Improperly Interfere

Durability → Committed Data Survives

■ Quick Placement Questions

- What is a transaction? → Logical unit of database work.
- Which ACID property means all-or-nothing? → Atomicity.
- Which property ensures committed changes survive a crash? → Durability.
- Which command permanently saves changes? → COMMIT.
- Which command undoes uncommitted changes? → ROLLBACK.
- What is a deadlock? → Transactions waiting for each other's resources.

FINAL 1-MINUTE REVISION — MODULES 10–13

Topic	Remember
Subquery	Query inside another query
Correlated Subquery	Depends on outer query row
EXISTS	Checks whether at least one row exists

Topic	Remember
1NF	Atomic values / no repeating groups
2NF	1NF + no partial dependency
3NF	2NF + no transitive dependency
BCNF	Every determinant is a super key
Functional Dependency	$X \rightarrow Y$: X determines Y
Partial Dependency	Part of composite key determines non-key attribute
Transitive Dependency	Non-key attribute determines another non-key attribute
Transaction	Logical unit of work
ACID	Atomicity, Consistency, Isolation, Durability
COMMIT	Save transaction
ROLLBACK	Undo uncommitted changes
SAVEPOINT	Mark rollback point
Deadlock	Transactions wait for each other

Placement strategy: Understand the definitions first. Then practice SQL subqueries, normalization problems, FD/key questions, and ACID/concurrency MCQs. These topics are frequently tested as concept-based questions.